Gradient Descent Methods over Multi-Class Logistic Regression

Juan Jose Baldelomar Cabrera 2181794, Davide Giardini 2202652,
Emiliano Salinas Arroyo 2185510, Sara Francesca Scully 2194423

Abstract

Multi-Class Logistic Regression is a widely used Machine Learning Algorithm with applications across both STEM studies as well as real-world industry applications. Today, implementations are straightforward with several options to choose from, especially in Python. Nevertheless, in order to study how this algorithm performs from a computational perspective, the available resources are limited. In this study, we implement a class-based optimisation Oracle that fixes input data to exploit the objective function's linear relations as well as a second one that caches the gradient's history. This implementations are fully developed as Numpy's computations to ensure reliability and efficiency, and they are used with three Gradient Descent Methods: Classic Gradient Descent and two BCGD methods with different block selection rules (Randomised and Gauss-Southwell). To analyse the performance, we provide several graphic measurements on the different properties of the algorithms, as well as a numerical results table to benchmark them against each other over the performance on three different datasets.

1 Introduction

In this study, we apply three unconstrained optimisation algorithms to address a multi-class logistic regression problem with the following form:

$$\min_{X \in \mathbb{R}^{d \times k}} \sum_{i=1}^{m} \left[-x_{b_i}^T a_i + \log \left(\sum_{c=1}^{k} \exp(x_c^T a_i) \right) \right] \tag{1}$$

The objective of this problem is to find an optimal parameter matrix $X \in \mathbb{R}^{d \times k}$ of class representations by minimising the negative log-likelihood.

In these functions, m represents the total number of training examples, $a_i \in \mathbb{R}^d$ denotes the feature vector for the i-th observation, and $b_i \in \{1, \ldots, k\}$ identifies the corresponding categorical labels. The parameter matrix X is composed of columns x_c, each representing the specific weights assigned to a particular class c.

In order to address this minimisation problem, we implemented three first-order optimisation algorithms. First, we apply Gradient Descent (GD), which is a standard iterative approach that updates the entire parameter matrix simultaneously in the direction of the steepest descent. Then, we explore Block-Coordinate Gradient Descent (BCGD) methods using two different selection rules. First, the Randomised Rule chooses to update a single block of the matrix X chosen uniformly at random during each iteration. The second rule, Gauss-Southwell, is a greedy approach that selects the row block corresponding to the largest gradient component at each iteration. To test the performance of these algorithms, we apply them to solve classification tasks over three datasets; a synthetic dataset consisting of transformations of Gaussian noise, the widely used MNIST dataset of images of digits, and the *Blood Cell Image Datset* available at

`https://www.kaggle.com/datasets/unclesamulus/blood-cells-image-dataset`

2 Formulation

For a Multi-Class Logistic Regression problem, consider a set of m random points $a_i \in \mathbb{R}^d$ sampled from an unknown distribution $\mathcal{D}$, each of them labelled by the respective random variable b_i that can take as values $1, 2, \cdots, k$. Let us assume that the log-probabilities of the labels are proportional to a linear combination of the entries of the random points given by a vector of unknown weights (x_b) specific for each label b. This relationship can be summarised by the following equation

$$\mathbb{P}(b_i = b | a_i = a) \propto \exp(x_b^T a).$$

Normalising this expression, we obtain the exact probabilities:

$$\mathbb{P}(b_i = b | a_i = a) = \frac{\exp(x_b^T a)}{\sum_{c=1}^{k} \exp(x_c^T a)}.$$

Taking X as the compilation of columns of the weight vectors, the previous equation provides us with a probabilistic model defined over X as a parameter. Therefore, we can obtain the maximum likelihood estimator of X, which maximises the probability of the evidence according to our model assumptions. We proceed first by obtaining the likelihood:

$$L(X) = \prod_{i=1}^{m} \frac{\exp(x_{b_i}^T a_i)}{\sum_{c=1}^{k} \exp(x_c^T a_i)}.$$

Notice now that maximising the likelihood is equivalent to minimising the negative log-likelihood given by the following equation:

$$f(X) = -\ell(X) = \sum_{i=1}^{m} \left[-x_{b_i}^T a_i + \log \left(\sum_{c=1}^{k} \exp(x_c^T a_i) \right) \right].$$

Therefore, Multi-Class Logistic Regression can be stated as a minimisation problem of the negative log-likelihood, and as such, we can apply gradient descent methods to numerically solve it. To proceed with this approach, we now need to compute the gradient with respect to X of our objective function. Let us fixate for a moment on the partial derivative with respect to an entry of X. Notice that

$$\begin{aligned}
\frac{\partial f(X)}{\partial x_{jb}} &= \frac{\partial}{\partial x_{jb}} \sum_{i=1}^{m} \left[-x_{b_i}^T a_i + \log \left(\sum_{c=1}^{k} \exp(x_c^T a_i) \right) \right] \\
&= \sum_{i=1}^{m} \left[-\frac{\partial}{\partial x_{jb}} (x_{b_i}^T a_i) + \frac{\partial}{\partial x_{jb}} \log \left(\sum_{c=1}^{k} \exp(x_c^T a_i) \right) \right] \\
&= \sum_{i=1}^{m} \left[-\frac{\partial}{\partial x_{jb}} (x_{b_i}^T a_i) + \frac{1}{\sum_{c=1}^{k} \exp(x_c^T a_i)} \left(\sum_{c=1}^{k} \frac{\partial}{\partial x_{jb}} \exp(x_c^T a_i) \right) \right].
\end{aligned}$$

Notice now that

$$\begin{aligned}
\frac{\partial}{\partial x_{jb}} (x_{b_i}^T a_i) &= \begin{cases} a_{ij} & \text{if } b_i = b \\ 0 & \text{otherwise.} \end{cases} \\
&= a_{ij} \; \mathbf{1}(b_i = b).
\end{aligned}$$

On a similar note,

$$\frac{\partial}{\partial x_{jb}} \exp(x_c^T a_i) = \begin{cases} a_{ij} \; \exp(x_c^T a_i) & \text{if } c = b \\ 0 & \text{otherwise,} \end{cases}$$

and therefore

$$\sum_{c=1}^{k} \frac{\partial}{\partial x_{jb}} \exp(x_c^T a_i) = a_{ij} \exp(x_b^T a_i).$$

Substituting these results on the partial derivative expression, we get

$$\frac{\partial f(X)}{\partial x_{jb}} = -\sum_{i=1}^{m} a_{ij} \left[\mathbf{1}(b_i = b) - \frac{\exp(x_b^T a_i)}{\sum_{c=1}^{k} \exp(x_c^T a_i)} \right].$$

For gradient computation, it remains only to compute these partial derivatives for all entries in X, but to do so efficiently to not increase the computational complexity unnecessarily.

2.1 Technical Considerations

It is well known that performing mathematical computations directly on Python is not advised as it is particularly slow. The safest approach to perform the computations is then to exploit the linear relations of the individual computations and then implement the calculations as operations between objects of a Python library specialised for linear algebra operations. Furthermore, as we will see in the following section, the Block-Coordinate-based gradient methods will consider the rows of X as blocks. Therefore, our gradient computations have to accommodate the restraint of individual rows.

Let $A \in \mathbb{R}^{m \times d}$ be the compilation in rows of the sample points a_i. Let $I \in \mathbb{R}^{m \times k}$, be an indicator matrix, where $I_{ib} = 1$ if the $i-$th sample is labelled b and $I_{ib} = 0$ otherwise. Let $T \in \mathbb{R}^{m \times k}$ be such that

$$\begin{aligned} T_{ib} &= \frac{\exp(a_i^T x_b)}{\sum_{c=1}^{k} \exp(a_i^T x_c)}. \\ &= \frac{(\exp(AX))_{ib}}{\sum_{c=1}^{k} (\exp(AX))_{ic}}. \end{aligned}$$

Notice that T is the **Softmax** transformation by rows of AX, which means

$$T = \begin{pmatrix} \text{Softmax}(a_1^T X) \\ \vdots \\ \text{Softmax}(a_m^T X) \end{pmatrix},$$

where, in general, for a given dimension n,

$$\begin{aligned} \text{Softmax} : \mathbb{R}^n &\to \mathbb{R}^n \\ x &\mapsto \left(\frac{\exp(x_i)}{\sum_{j=1}^{n} \exp(x_j)} \right)_{i=1}^{n}. \end{aligned}$$

Notice now that

$$\begin{aligned} [-A^T(I - T)]_{jb} &= -\sum_{i=1}^{m} a_{ij} \left[I_{ib} - T_{ib} \right]. \\ &= -\sum_{i=1}^{m} a_{ij} \left[\mathbf{1}(b_i = b) - \frac{\exp(x_b^T a_i)}{\sum_{c=1}^{k} \exp(x_c^T a_i)} \right]. \\ &= [\nabla_X f(X)]_{jb}. \end{aligned}$$

Therefore, we obtain

$$\nabla_X f(X) = -A^T(I - T). \tag{2}$$

On a similar note, we can compute the gradient with respect to a specific block (row of X):

$$\nabla_{x_j} f(X) = -A_j^T(I - T).$$

2.1.1 Objective Function Properties

Let us now study some properties of our objective function which will be useful for the required constraints of our optimization algorithms.

Definition 2.1 (Lipschitz Continuous Gradient)**.** *A function $f : \mathbb{R}^n \to \mathbb{R}$ has Lipschitz continuous gradient if there exists $L > 0$ such that*

$$\|\nabla f(x) - \nabla f(y)\| \leqslant L\|x - y\|, \qquad \forall x, y \in \mathbb{R}^n.$$

Theorem 1. *The Softmax function is 1-Lipschitz.*

Proof. This proof is omitted. □

Theorem 2. *The multiclass logistic regression loss has a continuous Lipschitz gradient.*

Proof. Let $T : \mathbb{R}^{d\times k} \to \mathbb{R}^d$ be such that

$$T(X) = \begin{pmatrix} \text{Softmax}(a_1^T X) \\ \vdots \\ \text{Softmax}(a_m^T X) \end{pmatrix}.$$

By equation 2, and the triangle inequality, given $X, H \in \mathbb{R}^{d\times k}$, we have

$$\begin{aligned} \|\nabla_X f(X + H) - f(X)\| &= \|A^T(T(X + H) - T(X))\|. \\ &\leqslant \|A\|_{\text{op}}\|T(X + H) - T(X)\|. \\ &\leqslant \|A\|_{\text{op}} \sum_{i=1}^{m} \|\text{Softmax}(a_i^T(X + H)) - \text{Softmax}(a_i^T X)\|. \end{aligned}$$

Notice now that by theorem 1 we have

$$\begin{aligned} \|\text{Softmax}(a_i^T(X + H)) - \text{Softmax}(a_i^T X)\| &\leqslant \|(a_i^T(X + H)) - (a_i^T X)\| \\ &= \|a_i^T H\| \\ &\leqslant \|H\|\|a_i\|, \end{aligned}$$

for $i = 1, \cdots, m$. Finally, the inequality continues as

$$\|\nabla_X f(X + H) - f(X)\| \leqslant \|A\|_{\text{op}} \left(\sum_{i=1}^{m} \|a_i\| \right) \|H\|.$$

Therefore, $\nabla_X f$ is $\|A\|_{\text{op}} \left(\sum_{i=1}^{m} \|a_i\|\right)$-Lipschitz. □

This proof can be generalised for the $j-$block gradient of f for rows as blocks.

2.1.2 Gauss-Southwell Cache Exploitation

One of the main advantages of the Gauss-Southwell selection rule is the fact that previously calculated scores can be cached in memory so that at each subsequent iteration there is no need to fully calculate AX.

To do this, a matrix $S \in \mathbb{R}^{(m\times k)}$ is initialised as

$$S = AX$$

The GS selection rule will give the j^* block (row) of X on which the update will be performed. We can call this vector $\Delta x \in \mathbb{R}^k$ and create a new matrix $\Delta X \in \mathbb{R}^{(m \times k)}$ by getting the dot product with the canonical vector $e_{j*} \in \mathbb{R}^d$, where only the j^*-th is 1 and every other is 0. Thus, we have

$$\Delta X = e_{j*} \Delta x^T$$

where the j^*-th row of this matrix is Δx and the rest is filled with zeros.

After one iteration, we can see the new parameters as $X^{\text{new}} = X + \Delta X$, and the updated scores as

$$\begin{aligned} S^{\text{new}} &= AX^{\text{new}}. \\ &= A(X + \Delta X). \\ &= AX + A\Delta X. \\ &= S + A(e_{j*} \Delta x^T). \\ &= S + (Ae_{j*}) \Delta x^T, \end{aligned}$$

where $Ae_{j*} \in \mathbb{R}^{(m \times 1)}$ is the j^*-th column of A.

In this way, instead of doing $\mathcal{O}(mkd)$ multiplications in every iteration, we reduce it to $\mathcal{O}(mk)$.

3 Algorithm

As stated above, we implemented 3 unconstrained optimisation algorithms: Gradient descent, BCGD with Randomised Rule, and BCGD with Gauss-Southwell Rule.

3.1 Gradient Descent

Algorithm 1 Gradient Descent Algorithm

Require: Initial parameter vector $x^{(0)}$, gradient oracle $\nabla f(x)$, step size α, tolerance ϵ, maximum iterations K

Ensure: Approximate solution $x^\star$

1: $k \leftarrow 0$
2: Initialise $x^{(k)} \leftarrow x^{(0)}$
3: **while** $k < K$ **do**
4: Compute gradient:

$$g^{(k)} \leftarrow \nabla f(x^{(k)})$$

5: Compute gradient norm:

$$\|g^{(k)}\|$$

6: **if** $\|g^{(k)}\| < \epsilon$ **then**
7: **break**
8: **end if**
9: Update:

$$x^{(k+1)} \leftarrow x^{(k)} - \alpha^{(k)} d^{(k)}$$

10: $k \leftarrow k + 1$
11: **end while**
12: **return** $x^{(k)}$

Gradient Descent finds an optimal vector $x^* \in \mathbb{R}^n$ that minimises a given objective function $f(x)$ by iteratively moving in the direction opposite to the gradient of the objective function, $\nabla f(x)$. Since the gradient indicates the direction of the steepest increase, when we step in the negative gradient direction ($-\nabla f(x)$, it produces the greatest local decrease in the loss function.

The process is presented in Algorithm 1, which begins with an initial parameter vector x_0 and repeatedly evaluates the gradient norm, $\|\nabla f(x_k)\|$, at each iteration k. This serves as a measure of how close the current iteration is to a stationary point. We impose a stopping tolerance threshold ϵ, such that when

$$\|\nabla f(x_k)\| < \epsilon,$$

the algorithm terminates. Under convexity assumptions, this stationary point corresponds to a global minimum. If the stopping criterion is not satisfied, the parameters are updated according to the gradient descent iteration:

$$x_{k+1} = x_k - \alpha \nabla f(x_k),$$

where $\alpha > 0$ is the learning rate (step size), controlling the magnitude of each update. Values that are too small may lead to slow convergence, whereas excessively large values may result in instability or divergence.

3.1.1 Conditions for Gradient Descent

To derive formal convergence bounds, the objective function must satisfy the appropriate smoothness assumptions. These assumptions ensure that each iteration decreases the objective function and that convergence can be established theoretically.

When the gradient is Lipschitz continuous, the descent lemma provides a quadratic upper bound on the objective function:

$$f(x) \leqslant f(y) + \nabla f(y)^T (x - y) + \frac{L}{2}\|x - y\|^2.$$

Using the descent lemma with the update rule $x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k)$, we obtain the following

$$f(x_{k+1}) - f(x_k) \leqslant -\frac{1}{2L}\|\nabla f(x_k)\|^2.$$

Whenever the gradient is non-zero, the objective function decreases monotonically

Theorem 3 (Convergence)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be a convex function with a Lipschitz continuous gradient having Lipschitz constant $L > 0$. The gradient method with a fixed step size $\alpha_k = \frac{1}{L}$ satisfies:*

$$f(x_{k+1}) - f(x^*) \leqslant \frac{2L\|x_1 - x^*\|^2}{k}$$

where x^* denotes a global minimiser of f and k is the iteration count.

3.2 Block-Coordinate Gradient Descent

In contrast to the Gradient Descent method, BCGD methods do not update the gradient in all directions. Instead, they take advantage of the possible relations between the coordinates of our objective variable, group them based on this criteria in blocks, and then optimise iteratively by calculating the active block gradient, one block at a time. To formalise this concept, assume that we are working with data on $\mathbb{R}^d$, and that the chosen blocks of coordinates are of size

$n_1, ..., n_b$ such that $n_1 + \cdots + n_b = d$. Then, let us construct the matrices $U_1, \cdots U_b$ such that $U_j \in \mathbb{R}^{d \times n_j}$ and that

$$I_d = \big(U_1 | \cdots | U_b.\big)$$

Notice that this construction assumes that the features in a block to be optimised are contiguous. Regarding implementation, this can be solved by performing a change of coordinates. For our minimisation problem in question, blocks are formed by rows of a matrix, which provides an already structured way to handle blocks from a computational perspective.

To implement BCGD methods over our objective functions, these have to satisfy certain constraints. Given a function $f : \mathbb{R}^d \to \mathbb{R}$, we say that f satisfies BCGD conditions if

1. f has a continuous Lipschitz gradient with constant L.
2. $f(\cdot, x_{-j})$ has a continuous Lipschitz gradient with constant L_i.

3.2.1 Randomized BCGD

One of the BCGD methods implemented in this work is the Randomised BCGD, which, to choose the block to optimise in each iteration, samples the block number from a particular distribution. The entire scheme of the process is referenced in algorithm 2. In our case, we choose the uniform distribution for choosing the block.

Algorithm 2 Randomized BCGD with tolerance

Require: Initial point $x^{(0)}$, objective function $f(x)$, coordinate blocks U_j, tolerance ε, maximum iterations K

Ensure: Approximate solution $x^\star$

$k \leftarrow 0$
while $k < K$ **do**
  Sample j_k from $1, \cdots, b$
  Compute $\nabla_{x_{j_k}} f(x_k)$
  Choose step size $\alpha^{(k)}$
  Update:

$$x^{(k+1)} \leftarrow x^{(k)} - \alpha^{(k)} U_{j_k} \nabla_{x_{j_k}} f(x_k)$$

  $k \leftarrow k + 1$
  if $\|U_{j_k} \nabla_{x_{j_k}} f(x_k)\| < \varepsilon$ **then**
    break
  end if
end while
return $x^{(k)}$

Theorem 4. *Given a function $f : \mathbb{R}^d \to \mathbb{R}$, such that f has a continuous Lipschitz gradient with constant L and $f(\cdot, x_{-j})$ has a continuous Lipschitz gradient with constant L_i, algorithm 2, with $\alpha^{(k)} = \frac{1}{L}$, satisfies:*

$$\mathbb{E}[f(x_{k+1}) - f(x^*)] \leqslant \frac{2LbR(x_1)^2}{k},$$

where

$$R(x_1) = \max\{\|x - x^*\|, f(x) \leqslant f(x_1) \text{ for } x \in \mathbb{R}^d\}.$$

Proof. The proof of this theorem is omitted as it was covered during lectures. □

Notice that by Theorem 2 and Theorem 4, we can ensure convergence for the Randomized BCGD algorithm over our objective function. Furthermore, they also provide a theoretical framework for testing our approach when implemented with the suggested step size.

3.2.2 Gauss-Southwell BCGD

The Gauss-Southwell BCGD method consists of selecting the block that yields the maximum block gradient norm of X. This results in a "greedy" approach since the updates will always be done over the directions of maximum descent per step. The main drawback is that it is still required to calculate the block gradient at each step in order to make the selection, but to counter this we can cache the scores as discussed in section 2.1.1.

The core structure for the implementation of GS-BCGD is shown in algorithm 3.

Algorithm 3 Gauss-Southwell BCGD with tolerance and cached scores

Require: Initial matrix $X^{(0)}$, objective function $f(x)$, feature matrix A, tolerance ε, maximum iterations K

Ensure: Approximate solution $X^\star$

$k \leftarrow 0$
$S \leftarrow AX^{(0)}$
while $k < K$ **do**
    Compute $\nabla f(X^{(k)})$ using $S^{(k)}$
    Pick block j^* such that $j^* = \text{Argmax}_{j\in\{1,\dots,d\}} \|\nabla_j f(X^{(k)})\|$
    Choose step size $\alpha_{j^*}^{(k)}$
    $\Delta X_{j^*} = -\alpha_{j^*}^{(k)} \nabla_{j^*} f(X^{(k)})$
    Update:

$$X_{j^*}^{(k+1)} \leftarrow X_{j^*}^{(k)} + \Delta X_{j^*}$$

$$S^{(k+1)} \leftarrow S^{(k)} + (Ae_{j^*})\Delta X_{j^*}$$

    $k \leftarrow k + 1$
    if $\|\nabla_{j^*} f(X^{(k)})\| < \varepsilon$ **then**
        break
    end if
end while
return $X^{(k)}$

3.3 Step Size Selection

In theory, the best choice for a fixed step size is $\alpha = 1/L$, as it is the largest possible step that still guarantees a monotonic decrease in the objective function. When choosing the step size of the descent, as each block is treated individually, to prevent any overshooting for the local minimum, it is necessary to select a proper step size. For this specific case, the selection of the Lipschitz constant L will depend on the features of A to properly counter these effects. To achieve this, we set L_j for each specific block as

$$L_j = \gamma \|Ae_j\|^2$$

with $\gamma = 0.25$ for the case of a logistic regression problem [1]

4 Results

4.1 Datasets

For a controlled baseline for these algorithms, we generated a synthetic dataset designed to simulate a controlled generative process, but that should be trivial to solve for a multi-class logistic regression task. For this purpose, we generate a matrix of regressors of 1000 examples and 1000 features, $A \in \mathbb{R}^{1000\times1000}$, sampling its entries from a standard normal distribution. Subsequently, we define a set of *ground truth* weights $X \in \mathbb{R}^{1000\times50}$, which map the regressors to their probability in each category. To reflect the inherent noise found in the data, we add a residual error matrix to the linear product E. The final categorical labels are derived through artificial probabilities $b = \text{argmax}(AX + E)$, and then applying the *argmax* function to obtain the most probable category.

On the other hand, the MNIST dataset is widely used as a benchmark for classification tasks, as it provides more complex inputs but it is still strongly structured. The dataset consists of 60,000 of 28×28 pixels of black and white pictures of digits from 0 to 9.

4.1.1 Blood Cell Image Dataset

To test our approaches on real-world data we use the Blood Cell Image dataset that contains 17092 high-resolution images of individual normal blood cells across 8 distinct classes. A primary challenge of this dataset is the class imbalance, with sample sizes ranging from 1214 (lymphocytes) to 3329 (neutrophils).

To facilitate efficient classification, images were first converted to greyscale and resized to 64×64 pixels. This preprocessing step reduced the feature space to a 4096-dimensional vector and eliminated the potential laboratory-specific biases, including the staining variations, forcing the model to prioritise cellular morphology and nuclear density.

After greyscale conversion, we analysed the statistical structure of the 4096-dimensional feature space. We observed a significant informational redundancy among pixels, with an average correlation coefficient of 0.1568. This dependency confirms that the neighbouring pixels are statistically dependent and introduce spatial noise. To address this, we used the covariance matrix

$$\Sigma = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(x_i - \bar{x})^T$$

to perform Principal Component Analysis (PCA). This process decorrelates the feature space which, in turn, extracts a more efficient and compressed representation of the image data

As a baseline, the labelled features were standardised ($\mu = 0$ and $\sigma^2 = 1$) to ensure numerical stability. We then applied PCA to the dataset to learn the projection matrix W. To evaluate the quality of the resulting feature space, we measured the mean Euclidean distance between the 8 class centroids ($D_{centroids}$):

$$D_{centroids} = \frac{2}{k(k-1)}\sum_{i<j}||\mathbf{c}_i - \mathbf{c}_j||_2$$

As a first test, using 20% of the data at first, a distance of 4.20 was obtained using this implementation. While PCA managed to decorrelate the pixels, the resulting feature space still exhibited significant class overlap, suggesting that PCA did not manage to sufficiently isolate the discriminative features needed to separate the classes.

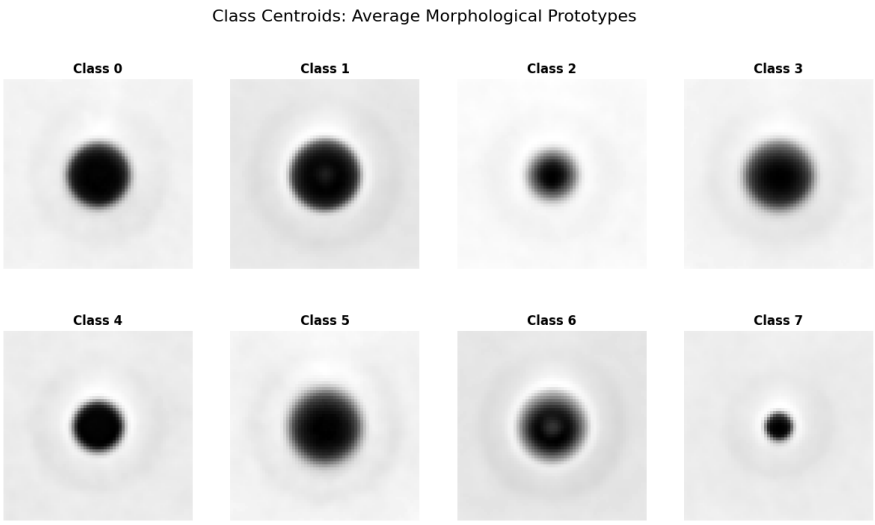


Figure 1: Class Centroids: Average morphology for each blood cell type (grayscaled). These prototypes represent the mean feature vector for each class in the original pixel space.

The morphological prototypes visualised in Figure 1 further illustrate this statistical redundancy. Each class centroid consists of a dark central structure surrounded by concentric halos which represent the localised variance that performing PCA attempts to summarise. However since these structures are so similar across different cell types, the global variance captured by PCA fails to emphasise the subtle morphological traits needed for clear class separation.

We tested different configurations to find the optimal balance between information retention and efficiency with the results shown in Table 1.

We then selected 150 components. In this optimised space, the average residual correlation dropped significantly to 1.46×10^{-15}, effectively achieving total feature independence. More importantly, the inter-class distance we obtained was 21.31.

Table 1: Variance Analysis

Components	**Variance**
50	55.1%
100	67.5%
150	74.2%
200	79.8%

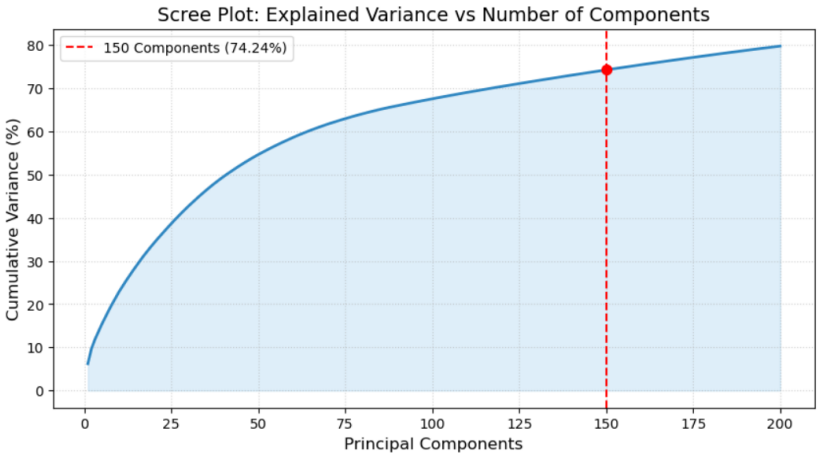


Figure 2: Cumulative variance vs. Principal components and our choice of 150.

4.1.2 Evaluation of Non-Linear Dimensionality Reduction

We explored other mechanisms different from the linear version of PCA over the Blood Cell Image dataset. One of our approaches was Kernel PCA (KPCA) with a Radial Basis Function (RBF) kernel:

$$K(x_i, x_j) = \exp(-\gamma||x_i - x_j||^2).$$

However, this approach yielded an extremely low inter-class distance (≈ 0.20 on a scale 0-1). While KPCA is effective for capturing non-linear manifolds, it tends to compress the data in this specific task. Given that our target model is a linear classifier (Logistic Regression), the linear PCA provided the most effective and interpretable input space.

4.2 Algorithm Performance

The results in Table 2 provide a performance comparison of the three optimisation algorithms discussed above. The data is tested across the three datasets: Synthetic, MNIST, and Blood Cell. For every entry in the table, we have reported three specific metrics from left to right: Training Accuracy, Testing Accuracy, and CPU Time (in seconds). All experiments were conducted under a fixed set of constraints, including a tolerance level of 10^{-4} and a maximum limit of 3000 iterations.

On the Synthetic dataset, all methods achieve high training accuracy. We obtain 0.965 for Gradient Descent, 0.943 for Randomised BCGD, and 0.986 for Gauss - Southwell BCGD. In contrast, we achieve an incredibly poor testing accuracy for all of the algorithms between 0.157 and 0.163. This imbalance between the high training accuracy and the low testing accuracy indicates severe overfitting. This can be explained by the way the data was generated and because both the features and the transformation weights are random, the dataset may lack

Table 2: Algorithm Performance. Training Accuracy (Tr), Testing Accuracy (Te), and CPU Time in seconds for each algorithm. Tolerance set to 10^{-4} with a maximum of 3000 iterations.

	Gradient Descent			**Randomized BCGD**			**GS BCGD**		
Dataset	Tr	Te	Time	Tr	Te	Time	Tr	Te	Time
Synthetic	0.965	0.157	349	0.943	0.163	282	0.986	0.158	242
MNIST	0.894	0.895	1330	0.892	0.896	1172	0.895	0.897	437
Blood Cell	0.622	0.610	9464	0.669	0.665	4931	0.699	0.684	4865

meaningful or stable class structure, whilst the added noise can dominate the signal. As a result, the models can fit the training samples well but fail to identify the patterns of the unseen data.

For the MNIST dataset, all three optimisation methods achieve nearly identical performance with training accuracies ranging from 0.892 to 0.895 and testing accuracies from 0.895 to 0.897. These results suggest that this dataset contains strong, learnable patterns that are consistently captured regardless of optimisation strategy. However, the major difference lies in computational efficiency: Gradient Descent requires 1330 CPU units, Randomised BCGD reduces this to 1172, and GS BCGD further lowers the runtime to 437. This indicates that GS BCGD converges much faster while maintaining equal or slightly improved predictive performance, making it especially effective for this high-dimensional image classification task.

The Blood Cell dataset presents a more challenging classification problem, reflected in lower overall accuracy values compared to MNIST. Gradient Descent achieves 0.622 training accuracy and 0.610 testing accuracy, while Randomised BCGD improves performance to 0.669 and 0.665, respectively. GS BCGD delivers the strongest results with 0.699 training accuracy and 0.684 testing accuracy. The reduced gap between training and testing accuracy across all methods suggests better generalisation than the Synthetic dataset, likely because Blood Cell images contain meaningful biological structure and feature relationships. In addition, both BCGD variants substantially reduce CPU time compared to Gradient Descent, with GS BCGD completing in 4865 CPU units versus 9464 for Gradient Descent

Overall, GS BCGD consistently provides the best balance between predictive accuracy and computational efficiency across the three datasets of various dimensions and complexity.

4.2.1 CPU time vs. Accuracy

In order to see the performance of our three methods we computed the CPU time at each iteration, then we obtained the accuracy of each iteration at the end of each process. We did

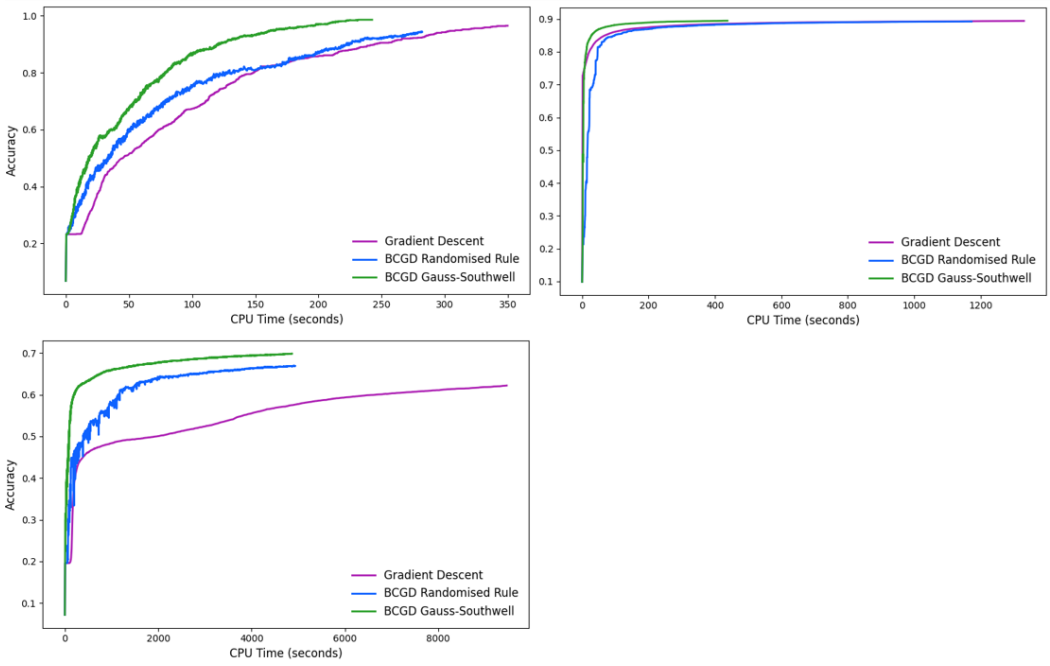


Figure 3: CPU Time vs. Training Accuracy. The datasets are presented in the order Synthetic, MNIST and Blood Cell (from left to right, top to bottom). Tolerance was set to 10^{-4} and maximum iterations to 3000.

this on the three datasets and Figure 3 shows the results of these runs.

Among the three datasets, the method that performed the best was BCGD with Gauss-Southwell selection rule. The performance of the three methods remained fairly similar over the Synthetic and MNIST datasets, yet for the Blood Cells dataset the Gradient Descent struggled to provide an optimal solution. This behaviour can be attributed to the fact that both the Synthetic and MNIST datasets are baseline dataset that have well-behaved properties, whereas the Blood Cell datasets come from a real world problem that is more difficult to get the underlying distribution from where the samples come from. This behaviour showcases the advantages that more elaborate algorithms gain when data start showing more complex patterns.

4.2.2 Iteration vs. Distance to Solution

Another way we tested the methods was to see how well they approximate the solution over the Synthetic dataset. We did this by getting the distance from the solution calculated in each iteration to the final solution obtained and the ground truth from where the data were generated, as shown in Figure 4.

We can see that the three of them arrive at their own solution at a similar rate, yet when comparing it to the real solution, both BCGD algorithms end up overfitting the data, especially with the Gauss-Southwell selection rule. In contrast, the normal Gradient Descent has a more robust approach that prevents this overfitting on the same number of iterations.

4.2.3 Algorithmic Breakdown Point

Another point that we considered during our tests was the impact of a constant step size over the methods to find out at which point each of the methods breaks down, and more generally, how they perform when limited knowledge of the data is present. This approach only makes sense when the data is distributed equally across dimensions, and therefore synthetic data was used to proceed with this measurement. Taking into account computational burden, we chose to use a smaller Synthetic dataset consisting of 500 samples, 100 features, and 50 categories. We ran the experiment until convergence was achieved or at the maximum number of iterations. We discretely varied the step size from 10^{-3} to 10, distributed equally from a logarithmic perspective, setting a tolerance for the gradient norm at 1 and a maximum number of iterations of 10^4, and the results for each method were no longer mapped when a gradient explosion occurs. The results are shown in Figure 5.

The BCGD method with the Gauss-Southwell selection rule has a high sensitivity for the step size; with both small and large step sizes it maxes out the number of iterations required

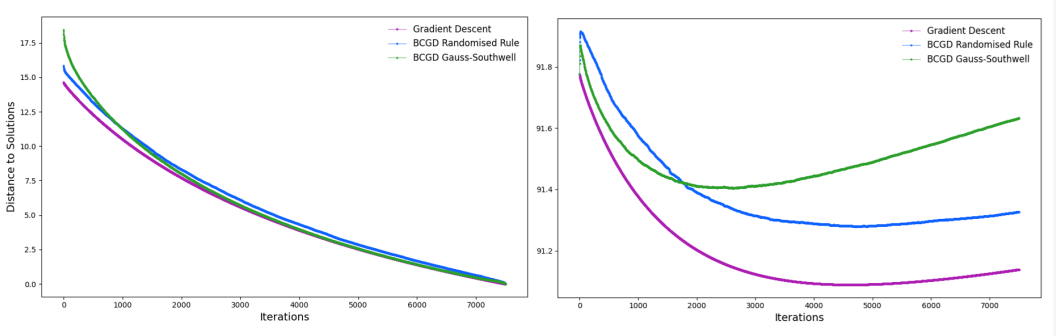


Figure 4: Iterations vs. Distance to solutions over the Synthetic dataset. On the left, the distance to the obtained solution. On the right, the distance towards the ground truth.

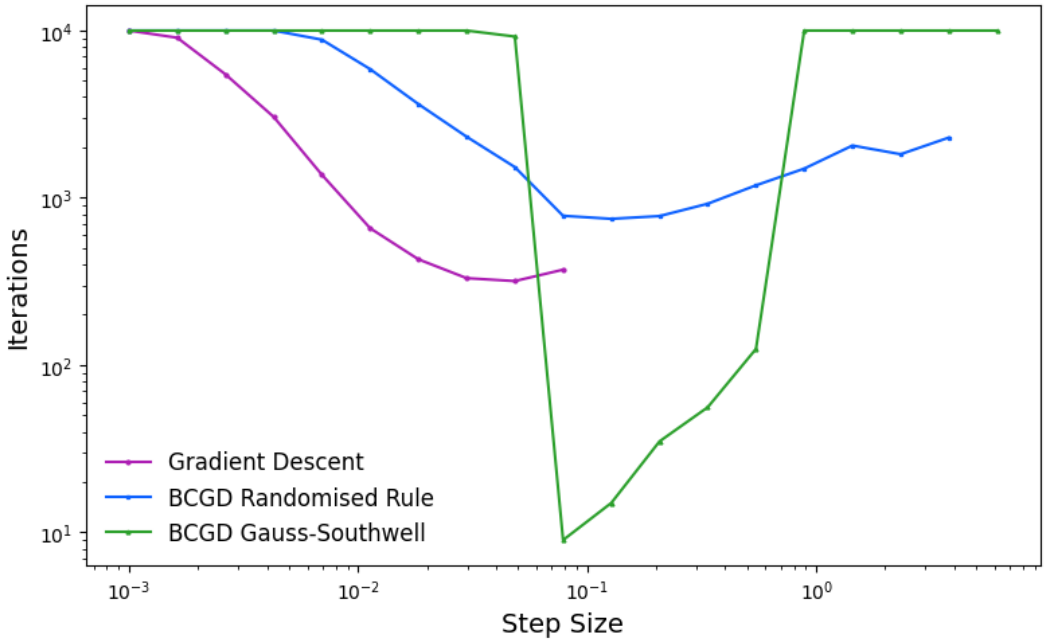


Figure 5: Step Size vs. Iterations: Over Synthetic data of 50 categories, 100 features and 500 examples, the graph shows the number of iterations until convergence (or halted at the set max iterations) with a tolerance set to 1 and maximum iterations set to 10,000. The scales are logarithmic, the step sizes go from 0.001 to 10, and when gradient explosion occurs the results are no longer mapped.

to arrive to a solution under the given convergence tolerance. This is due to the fact that as it always selects the block with the steepest descent, the curvature of the block might not match the Lipschitz constant associated with the step size α casing to either overshoot the minimum or not being "aggressive" enough to fully arrive at the solution; yet when the step size is close to the average of $1/\bar{L}_j$ this makes the algorithm achieve a solution almost 100 faster than the other two methods. This means that in order to fully take advantage of this method, it is required to use a good approximation of the Lipschitz constant.

On the other hand, both the Gradient Descent and the Randomised BCGD appear to be more resilient to selection of the step size. In the case of the BCGD with the Randomised selection rule, this might be attributed to the fact that even though the step size is not closely related to the curvature of the chosen block, the randomised selection rule might yield a block that does not penalise a lot this wrong choice of step size and at the long run evens out. For the Gradient Descent, this method does not take into account just a single block but rather the whole domain, yet the gradient explodes sooner since it does not have the "dampening" behaviour provided by updating only one block at a time.

5 LLM Usage Disclosure Statement

Through this work LLMs were not used to explicitly write any of the code's logic or for the writing of any section of the report. LLMs were used only as a guide on how to use certain Python libraries, but none of the code on the repository was written by an LLM. Similarly, none of the theorems in the report or their respective proofs was written using an LLM.

References

[1] Julie Nutini, Mark Schmidt, Issam H. Laradji, Michael Friedlander, and Hoyt Koepke. Coordinate descent converges faster with the gauss-southwell rule than random selection. *arXiv:1506.00552v2*, 2015. Revised 2018.